



i want to achieve hierarchical nested table with 3 levels of depth for my shipping ERP application where line of Item will be first level and second level be Sourcing summary and if we expand sourcing it will show list of sourcing and also another second level for all level 1 Line of item is tariff summary and if we expand tariff summary we will view list of tariff records note the important point is each level of the hierarchy table has different set of columns and different column count how to achieve this kind of ui in angular give me a correct and working solution

For a 3-level hierarchical nested table in your shipping ERP application, the **Syncfusion Angular Grid** ([@syncfusion/ej2-angular-grids](#)) or **Angular Material nested MatTable** are both valid solutions. However, since each level in your case has **different columns and column counts**, Syncfusion's **Hierarchy Grid** offers a clean, declarative, and fully working approach out-of-the-box.

Recommended: Syncfusion Angular Hierarchy Grid

Syncfusion's `ejs-grid` component allows **n-level hierarchical nesting**, with **different columns at each level**, and expandable sections such as *Sourcing Summary* and *Tariff Summary*.

Step-by-step Implementation

1. Install dependencies:

```
npm install @syncfusion/ej2-angular-grids
npm install @syncfusion/ej2-base @syncfusion/ej2-data
```

2. Import modules in `app.module.ts`:

```
import { GridModule, DetailRowService } from '@syncfusion/ej2-angular-grids';
import { BrowserModule } from '@angular/platform-browser';
```

```
import { NgModule } from '@angular/core';
import { AppComponent } from './app.component';

@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule, GridModule],
  providers: [DetailRowService],
  bootstrap: [AppComponent],
})
export class AppModule {}
```

3. Define 3-level nested grids in app.component.ts:

```
import { Component, OnInit } from '@angular/core';
import { GridModel } from '@syncfusion/ej2-angular-grids';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
})
export class AppComponent implements OnInit {
  public parentData?: object[];
  public sourcingChildGrid?: GridModel;
  public tariffChildGrid?: GridModel;
  public lineItemChildGrid?: GridModel;
  public grandChildGrid?: GridModel;

  ngOnInit() {
    this.parentData = [
      { LineID: 1, ItemName: 'Container', Quantity: 10, TariffCount: 2 },
      { LineID: 2, ItemName: 'Ship Crate', Quantity: 5, TariffCount: 3 },
    ];

    // Level 2-A: Sourcing summary child grid
    this.sourcingChildGrid = {
      queryString: 'LineID',
      columns: [
        { field: 'SourcingType', headerText: 'Source Type', width: 150 },
        { field: 'Supplier', headerText: 'Supplier', width: 150 },
        { field: 'Status', headerText: 'Status', width: 120 },
      ],
      dataSource: [
        { LineID: 1, SourcingType: 'Warehouse', Supplier: 'Alpha', Status: 'Active' },
        { LineID: 1, SourcingType: 'Local', Supplier: 'Beta', Status: 'Pending' },
        { LineID: 2, SourcingType: 'Export', Supplier: 'Gamma', Status: 'Approved' },
      ],
      childGrid: {
        queryString: 'Supplier', // Level 3: sourcing detail records
        columns: [
          { field: 'PartNo', headerText: 'Part Number', width: 120 },
          { field: 'Batch', headerText: 'Batch ID', width: 100 },
          { field: 'ShipDate', headerText: 'Ship Date', width: 130 },
        ],
        dataSource: [
          { Supplier: 'Alpha', PartNo: 'A123', Batch: 'B001', ShipDate: '2025-09-12' },

```

```

        { Supplier: 'Beta', PartNo: 'B999', Batch: 'B002', ShipDate: '2025-10-01' },
        { Supplier: 'Gamma', PartNo: 'G777', Batch: 'B003', ShipDate: '2025-10-05' },
    ],
    },
};

// Level 2-B: Tariff summary grid
this.tariffChildGrid = {
    queryString: 'LineID',
    columns: [
        { field: 'TariffType', headerText: 'Tariff Type', width: 120 },
        { field: 'Charge', headerText: 'Charge', format: 'C2', width: 120 },
        { field: 'Currency', headerText: 'Currency', width: 80 },
    ],
    dataSource: [
        { LineID: 1, TariffType: 'Import', Charge: 1200, Currency: 'USD' },
        { LineID: 1, TariffType: 'Port', Charge: 300, Currency: 'USD' },
        { LineID: 2, TariffType: 'Export', Charge: 900, Currency: 'USD' },
    ],
    },
};
}
}

```

4. Define the Template in app.component.html:

```

<ejs-grid
  [dataSource]="parentData"
  [childGrid]="sourcingChildGrid"
  [allowPaging]="false"
  height="300px"
>
  <e-columns>
    <e-column field="LineID" headerText="Line ID" width="80"></e-column>
    <e-column field="ItemName" headerText="Item Name" width="150"></e-column>
    <e-column field="Quantity" headerText="Quantity" width="100"></e-column>
  </e-columns>
</ejs-grid>

<hr />

<ejs-grid
  [dataSource]="parentData"
  [childGrid]="tariffChildGrid"
  [allowPaging]="false"
  height="300px"
>
  <e-columns>
    <e-column field="LineID" headerText="Line ID" width="80"></e-column>
    <e-column field="ItemName" headerText="Item Name" width="150"></e-column>
    <e-column field="TariffCount" headerText="No. of Tariffs" width="120"></e-column>
  </e-columns>
</ejs-grid>

```

Why This Works Best

- Each table can have its **own column configuration** and **different child relationships**.^[1]
- Supports **3+ levels of nesting** via chained `childGrid` objects.^[1]
- Allows **dynamic data loading** and independent **expand/collapse** control per level.^[1]
- Fully TypeScript-configurable and works seamlessly in Angular 15–17.

Alternate Option

If you prefer **Material Design**, a working triple-nested MatTable demo exists on StackBlitz showing expandable sub-tables for 3 levels (`angular-nested-mat-table-triplenested`). However, Syncfusion's approach is more robust for **ERP-grade tabular hierarchy needs**, especially with **multiple column sets per level**.^[2]

✱

is ej2-angular-grids free to use or else paid version

The `@syncfusion/ej2-angular-grids` package is **not fully free and open source** — it is a **commercial (paid) component** from Syncfusion's Essential JS 2 suite.^{[21] [22] [23]}

However, there are **two licensing options** depending on your use case:

1. Community License (Free for eligible users)

Syncfusion provides a **free community license** for:

- Individual developers.
- Small companies with **annual gross revenue below \$1 million USD**.

This license includes **all Syncfusion components**, **no feature restrictions**, and **lifetime updates**, as long as eligibility criteria are met.^[22]

You can register for it here:

<https://www.syncfusion.com/products/communitylicense>.

2. Paid License (Commercial Use)

For enterprises or revenue-generating organizations exceeding the eligibility limits, you must purchase a **commercial license**.

Paid licenses give:

- Commercial usage rights.
- 24/7 priority support.
- Access to source code and advanced enterprise-level integrations.^{[24] [25] [21]}

Summary Table

License Type	Cost	Eligibility	Notes
Community Edition	Free	Individuals or organizations under \$1M annual revenue [22]	Full features, no restrictions
Commercial License	Paid (annual)	Larger businesses or commercial use [21]	Includes support, updates, and legal use

In short, **Syncfusion's ej2-angular-grids is paid**, but **you can use it completely free under their community license** if your organization meets the eligibility criteria. [\[21\]](#) [\[22\]](#)



i cant use this unless i have community or paid license of sync fusion is it so

Yes — you **cannot legally or fully use** the @syncfusion/ej2-angular-grids package **without a valid Syncfusion license key**, whether from a **Community License**, **Trial License**, or **Paid License**.

Key Points

- Starting from **version 20.1.0.47 (2022 Volume 1)**, all **Essential JS2 (EJ2)** products, including **ej2-angular-grids**, require a **license key registered in your code**. [\[31\]](#) [\[32\]](#)
- A **valid license key** can only be obtained via:
 - A **30-day evaluation/trial license**.
 - A **free community license** (if eligible).
 - A **commercial license** (for enterprise or paid usage). [\[33\]](#) [\[34\]](#)
- Without registering a valid key, your app will show a **license validation popup or console message** such as
"This application was built using a trial version of Syncfusion Essential Studio. Please include a valid license key." [\[35\]](#) [\[31\]](#)
- The license is **checked locally (offline)** and does not contact Syncfusion's servers, but you still must include a valid key in main.ts using:

```
import { registerLicense } from '@syncfusion/ej2-base';
registerLicense('YOUR_LICENSE_KEY');
```

In summary:

You **can't use ej2-angular-grids** in production or long-term development **without** one of the following:

- A **free community license** (if eligible under revenue or personal-use criteria).
- A **30-day trial key** (for evaluation).
- A **commercial paid license** (for enterprise or commercial projects).

Otherwise, the grid will function only with **trial limitations and license warnings**. ^[34] ^[31] ^[35]



1. <https://ej2.syncfusion.com/angular/documentation/grid/hierarchy-grid>
2. <https://stackblitz.com/edit/angular-nested-mat-table-triplenested?file=app%2Ftable-expandable-rows-example.html>
3. <https://stackblitz.com/edit/angular-nested-mat-table?file=app%2Ftable-expandable-rows-example.ts>
4. https://www.reddit.com/r/Angular2/comments/ehsmey/material_table_with_expandable_rows_how_to/
5. <https://www.ag-grid.com/angular-data-grid/tree-data-paths/>
6. <https://material.angular.dev/components/table/examples>
7. <https://www.ag-grid.com/angular-data-grid/tree-data-opening-groups/>
8. <https://www.infragistics.com/products/ignite-ui-angular/angular/components/grid/multi-column-headers>
9. <https://www.youtube.com/watch?v=5aDFxFIGvSc>
10. <https://angular.delldesignsystem.com/2.25.0/index.html?path=%2Fdocs%2Fcomponents-table--expandable-rows>
11. <https://js.devexpress.com/Angular/Demos/WidgetsGallery/Demo/DataGrid/MultiRowHeadersBands/>
12. <https://www.material-react-table.com/docs/guides/expanding-sub-rows>
13. <https://www.ag-grid.com/angular-data-grid/master-detail-nesting/>
14. <https://www.infragistics.com/products/ignite-ui-angular/angular/components/hierarchicalgrid/hierarchical-grid>
15. <https://stackoverflow.com/questions/68671588/angular-table-with-expandable-row-hierarchy-structure>
16. <https://www.ag-grid.com/angular-data-grid/grouping-multiple-group-columns/>
17. <https://www.tutorialsteacher.com/articles/angular-grid-examples>
18. <https://stackoverflow.com/questions/69189977/angular-11-table-with-two-nested-expandable-rows>
19. <https://www.geeksforgeeks.org/angular-js/angular-primeng-table-expandable-rows/>
20. <https://js.devexpress.com/Angular/Demos/WidgetsGallery/Demo/TreeView/HierarchicalDataStructure/>
21. <https://www.npmjs.com/package/@syncfusion/ej2-angular-grids/v/20.2.38>
22. <https://www.syncfusion.com/forums/136792/is-essential-js-2-for-angular-is-open-source-or-it-is-paid>
23. <https://github.com/syncfusion/ej2-angular-ui-components>
24. <https://www.syncfusion.com/sales/unlimitedlicense>
25. <https://www.syncfusion.com/angular-components>

26. <https://www.syncfusion.com/angular-components/angular-data-grid>
27. <https://ej2.syncfusion.com/angular/documentation/grid/editing/batch-editing>
28. <https://ej2.syncfusion.com/angular/documentation/grid/getting-started>
29. <https://codesandbox.io/examples/package/@syncfusion/ej2-angular-grids>
30. <https://ej2.syncfusion.com/angular/documentation/grid/editing/in-line-editing>
31. <https://ej2.syncfusion.com/angular/documentation/licensing/overview>
32. <https://ej2.syncfusion.com/documentation/licensing/overview>
33. <https://www.syncfusion.com/products/communitylicense>
34. <https://ej2.syncfusion.com/angular/documentation/licensing/license-key-registration>
35. <https://www.syncfusion.com/blogs/post/introducing-license-key-validation-for-the-essential-js-2-platforms>
36. <https://www.syncfusion.com/forums/166182/licensing-for-using-ej2-angular-diagrams-npm-package>
37. <https://www.syncfusion.com/forums/157074/what-do-i-do-with-my-license>
38. <https://ej2.syncfusion.com/angular/documentation/licensing/license-key-generation>
39. <https://ej2.syncfusion.com/angular/documentation/licensing/licensing-troubleshoot>
40. <https://help.syncfusion.com/common/essential-studio/licensing/how-to-register-in-an-application>